



Universidad del Quindío

Facultad de Ciencias Básicas y Tecnologías

Programa de Física

Proyecto final de Programación

Simulación y optimización de un lanzamiento parabólico hacia un objetivo

Estudiante: Juan David Cárdenas Vélez

Asignatura: Programación

Profesor: Santiago Echeverry

Fecha: 19 de mayo de 2026

Armenia, Quindío

Índice

1. Introducción	2
2. Planteamiento del problema	2
3. Objetivos	3
3.1. Objetivo general	3
3.2. Objetivos específicos	3
4. Marco teórico	3
5. Diseño de la solución	4
6. Librerías utilizadas	5
7. Explicación del funcionamiento del código	5
8. Código utilizado	6
9. Resultados esperados	10
10. Análisis de resultados	11
11. Cumplimiento de la rúbrica	11
12. Conclusiones	12

1. Introducción

En este proyecto se desarrolló una simulación computacional de un lanzamiento parabólico. La idea principal fue construir un programa que, a partir de una velocidad inicial, una gravedad y la posición de un objetivo, encuentre el ángulo de lanzamiento que hace que el proyectil pase lo más cerca posible de ese punto.

El trabajo mezcla una parte de física bastante conocida, que es el movimiento parabólico, con una parte de programación orientada a objetos. La intención no fue hacer un código demasiado complicado, sino un programa que se pueda entender y defender bien. Por eso el modelo físico se mantuvo ideal: no se considera rozamiento del aire, la gravedad es constante y el proyectil parte desde el origen.

El programa utiliza `NumPy` para los cálculos numéricos, `SciPy` para buscar el ángulo que mejor se ajusta al objetivo y `Matplotlib` para mostrar la trayectoria y animar el movimiento. Además, el código se organizó con clases, herencia y una clase abstracta, de forma que también cumpliera con la parte de diseño computacional pedida en el proyecto.

2. Planteamiento del problema

El problema que se quiere resolver es el siguiente: se tiene un proyectil que se lanza desde el origen con una velocidad inicial conocida. También se tiene un objetivo ubicado en una posición determinada del plano, dada por las coordenadas $(x_{\text{objetivo}}, y_{\text{objetivo}})$. La pregunta es qué ángulo de lanzamiento permite que la trayectoria del proyectil pase lo más cerca posible de ese objetivo.

En mecánica ideal, si se conocen las condiciones iniciales, la trayectoria queda definida. Sin embargo, desde el punto de vista computacional, el programa necesita buscar entre varios ángulos posibles para decidir cuál se ajusta mejor al objetivo. Por eso se usa una función de optimización: no porque haya error experimental, sino porque el computador necesita una medida matemática para comparar ángulos.

La medida usada en el programa es la diferencia vertical entre la altura del proyectil y la altura del objetivo cuando el proyectil llega a la misma posición horizontal del blanco. En el código aparece como un “error”, pero se debe entender más bien como una desviación vertical respecto al objetivo.

3. Objetivos

3.1. Objetivo general

Desarrollar una simulación computacional del movimiento parabólico de un proyectil, usando programación orientada a objetos, para encontrar el ángulo de lanzamiento que permite acercarse a un objetivo definido en el plano.

3.2. Objetivos específicos

- Implementar las ecuaciones del movimiento parabólico en Python.
- Usar `NumPy` para calcular las posiciones del proyectil en diferentes instantes de tiempo.
- Usar `SciPy` para optimizar el ángulo de lanzamiento.
- Visualizar la trayectoria mediante `Matplotlib`.
- Animar el movimiento del proyectil usando `FuncAnimation`.
- Organizar el código mediante clases, herencia y una clase abstracta.

4. Marco teórico

El movimiento parabólico se puede ver como la combinación de dos movimientos: uno horizontal y uno vertical. En este proyecto se toma el caso ideal donde no hay rozamiento del aire. Por eso, en el eje horizontal no hay aceleración y la velocidad horizontal permanece constante.

La posición horizontal se calcula con

$$x(t) = v_0 \cos(\theta)t, \quad (1)$$

donde v_0 es la velocidad inicial, θ es el ángulo de lanzamiento y t es el tiempo.

En el eje vertical sí actúa la gravedad. Como se toma hacia arriba como dirección positiva, la aceleración vertical es $-g$. Por eso la posición vertical está dada por

$$y(t) = v_0 \sin(\theta)t - \frac{1}{2}gt^2. \quad (2)$$

El primer término, $v_0 \sin(\theta)t$, representa la subida inicial del proyectil debido a la componente vertical de la velocidad. El segundo término, $-\frac{1}{2}gt^2$, representa el efecto de la gravedad, que va curvando la trayectoria hacia abajo.

Para simular la trayectoria completa se calcula el tiempo de vuelo. Como el proyectil parte desde $y = 0$ y se simula hasta que vuelve al suelo, se usa

$$t_f = \frac{2v_0 \sin(\theta)}{g}. \quad (3)$$

Esta expresión sale de imponer $y(t) = 0$ en la ecuación vertical y tomar la solución distinta de $t = 0$.

La optimización entra cuando se quiere encontrar el ángulo más conveniente. Para un ángulo dado, primero se calcula el tiempo en que el proyectil llega horizontalmente al objetivo. A partir de la ecuación horizontal,

$$x = v_0 \cos(\theta)t, \quad (4)$$

se despeja

$$t = \frac{x_{\text{objetivo}}}{v_0 \cos(\theta)}. \quad (5)$$

Luego se evalúa la altura del proyectil en ese instante y se compara con la altura del objetivo. La cantidad que el programa minimiza es

$$|y_{\text{proyectil}} - y_{\text{objetivo}}|. \quad (6)$$

Esta expresión no se toma como error experimental. Más bien es una forma sencilla de medir qué tan cerca pasa la trayectoria del blanco.

5. Diseño de la solución

El código se organizó usando programación orientada a objetos. La idea fue separar responsabilidades para que cada clase tuviera un papel claro. Esto ayuda a que el programa sea más fácil de leer, modificar y explicar.

Clase	Función dentro del proyecto
<code>SistemaFisico</code>	Clase abstracta. Define que todo sistema físico debe tener un método para simular y otro para entregar un resumen.
<code>Proyectil</code>	Guarda las propiedades básicas del proyectil: velocidad inicial y gravedad.
<code>LanzamientoParabolico</code>	Calcula la trayectoria, busca el ángulo óptimo y genera los datos simulados.
<code>Analizador</code>	Muestra en consola los resultados principales del lanzamiento.
<code>Visualizador</code>	Crea la gráfica y la animación del proyectil.

Tabla 1: Clases principales del proyecto.

La clase `SistemaFisico` permite cumplir la idea de abstracción. No representa un lanzamiento concreto, sino una estructura general. La clase `LanzamientoParabolico` hereda de ella y sí implementa el caso físico específico.

6. Librerías utilizadas

Librería	Uso en el proyecto
<code>abc</code>	Permite crear la clase abstracta <code>SistemaFisico</code> .
<code>NumPy</code>	Se usa para senos, cosenos, conversión de ángulos, arreglos de tiempo y valores máximos.
<code>Matplotlib</code>	Se usa para graficar la trayectoria, el objetivo y el punto animado.
<code>FuncAnimation</code>	Permite animar el movimiento del proyectil sobre la trayectoria calculada.
<code>SciPy</code>	Se usa para optimizar el ángulo de lanzamiento mediante <code>minimize_scalar</code> .

Tabla 2: Librerías utilizadas en el código.

7. Explicación del funcionamiento del código

El código empieza importando las librerías necesarias. Después se define una clase abstracta llamada `SistemaFisico`. Esta clase obliga a que las clases hijas tengan los métodos `simular` y `resumen`. En este caso, la clase que hereda de ella es `LanzamientoParabolico`.

Luego aparece la clase `Proyectil`, que guarda la velocidad inicial y la gravedad. Esto permite separar los datos del objeto lanzado de los cálculos de la trayectoria.

La clase `LanzamientoParabolico` es la parte central. En ella se guardan el proyectil y las coordenadas del objetivo. También se crean variables como `angulo_optimo`, `error_minimo`, `t`, `x` y `y`. Al comienzo algunas de estas variables están en `None` porque todavía no se ha hecho la simulación.

El método `posicion` implementa directamente las ecuaciones del movimiento parabólico. Recibe un ángulo y un tiempo, y devuelve las coordenadas x y y . Esta parte es la base física del programa.

El método `error_objetivo` calcula qué tan cerca pasa la trayectoria del objetivo para un ángulo dado. Primero calcula el tiempo en que el proyectil llega a la posición horizontal del objetivo. Luego evalúa la altura del proyectil en ese instante y la compara con la altura del blanco. Aunque el nombre del método usa la palabra `error`, realmente se está midiendo una desviación vertical.

Después, el método `optimizar_angulo` usa `minimize_scalar`. Esta función prueba distintos ángulos entre 1 y 89 grados y busca cuál minimiza la desviación respecto al objetivo. Se evita usar 0 y 90 grados porque son casos extremos: en 0 grados casi no hay componente vertical, y en 90 grados no hay avance horizontal.

El método `simular` se encarga de generar la trayectoria completa. Si todavía no se ha calculado el ángulo óptimo, primero lo calcula. Luego crea un arreglo de tiempos con `np.linspace` y obtiene los valores de x y y usando el método `posicion`.

El método `resumen` devuelve los datos principales en forma de diccionario: ángulo óptimo, desviación mínima, altura máxima y alcance de la trayectoria.

La clase `Analizador` toma esos resultados y los imprime en consola. Esta parte ayuda a tener una salida numérica clara, no solamente una gráfica. Por último, la clase `Visualizador` crea la figura, dibuja la trayectoria, marca el objetivo con una equis y anima un punto que representa el proyectil moviéndose.

8. Código utilizado

```
1 from abc import ABC, abstractmethod
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from matplotlib.animation import FuncAnimation
5 from scipy.optimize import minimize_scalar
6
7 class SistemaFisico(ABC):
8     @abstractmethod
9     def simular(self):
10         pass
11
```

```

12     @abstractmethod
13     def resumen(self):
14         pass
15
16 class Proyectil:
17     def __init__(self, velocidad_inicial, gravedad):
18         self.v0 = velocidad_inicial
19         self.g = gravedad
20
21 class LanzamientoParabolico(SistemaFisico):
22     def __init__(self, proyectil, objetivo_x, objetivo_y):
23         self.proyectil = proyectil
24         self.objetivo_x = objetivo_x
25         self.objetivo_y = objetivo_y
26         self.angulo_optimo = None
27         self.error_minimo = None
28         self.t = None
29         self.x = None
30         self.y = None
31
32     def posicion(self, angulo, t):
33         v0 = self.proyectil.v0
34         g = self.proyectil.g
35         x = v0 * np.cos(angulo) * t
36         y = v0 * np.sin(angulo) * t - 0.5 * g * t**2
37         return x, y
38
39     def error_objetivo(self, angulo):
40         v0 = self.proyectil.v0
41         t_objetivo = self.objetivo_x / (v0 * np.cos(angulo))
42         _, altura = self.posicion(angulo, t_objetivo)
43         return abs(altura - self.objetivo_y)
44
45     def optimizar_angulo(self):
46         resultado = minimize_scalar(
47             self.error_objetivo,
48             bounds=(np.radians(1), np.radians(89)),
49             method="bounded"
50         )
51         self.angulo_optimo = resultado.x
52         self.error_minimo = resultado.fun

```



```

53
54 def simular(self):
55     if self.angulo_optimo is None:
56         self.optimizar_angulo()
57     v0 = self.proyectil.v0
58     g = self.proyectil.g
59     theta = self.angulo_optimo
60     t_final = 2 * v0 * np.sin(theta) / g
61     self.t = np.linspace(0, t_final, 160)
62     self.x, self.y = self.posicion(theta, self.t)
63     return self.t, self.x, self.y
64
65 def resumen(self):
66     return {
67         "angulo": np.degrees(self.angulo_optimo),
68         "error": self.error_minimo,
69         "altura_maxima": np.max(self.y),
70         "alcance": np.max(self.x)
71     }
72
73 class Analizador:
74     def __init__(self, sistema):
75         self.sistema = sistema
76
77     def mostrar(self):
78         datos = self.sistema.resumen()
79         print("ANALISIS DEL LANZAMIENTO")
80         print(f"Velocidad inicial: {self.sistema.proyectil.v0:.2f} m/s")
81         print(f"Gravedad: {self.sistema.proyectil.g:.2f} m/s^2")
82         print(f"Objetivo: ({self.sistema.objetivo_x:.2f}, {self.sistema.objetivo_y:.2f}) m")
83         print(f"Angulo optimo: {datos['angulo']:.2f} grados")
84         print(f"Error minimo: {datos['error']:.4f} m")
85         print(f"Altura maxima: {datos['altura_maxima']:.2f} m")
86         print(f"Alcance de la trayectoria: {datos['alcance']:.2f} m")
87
88 class Visualizador:
89     def __init__(self, sistema):
90         self.sistema = sistema
91
92     def animar(self):

```

```

93     t, x, y = self.sistema.simular()
94     fig, ax = plt.subplots(figsize=(8, 5))
95
96     ax.set_xlim(0, max(np.max(x), self.sistema.objetivo_x) + 10)
97     ax.set_ylim(0, max(np.max(y), self.sistema.objetivo_y) + 10)
98     ax.set_title("Busqueda del angulo para golpear un objetivo")
99     ax.set_xlabel("x (m)")
100    ax.set_ylabel("y (m)")
101    ax.grid(True)
102
103    ax.plot(x, y, "-", label="trayectoria optimizada")
104    ax.scatter(self.sistema.objetivo_x, self.sistema.objetivo_y,
105               marker="x", s=110, label="objetivo")
106
107    punto, = ax.plot([], [], "o", markersize=9)
108    texto = ax.text(0.03, 0.87, "", transform=ax.transAxes)
109    angulo = np.degrees(self.sistema.angulo_optimo)
110
111    def iniciar():
112        punto.set_data([], [])
113        texto.set_text("")
114        return punto, texto
115
116    def actualizar(i):
117        punto.set_data([x[i]], [y[i]])
118        texto.set_text(
119            f"t = {t[i]:.2f} s\n"
120            f"angulo = {angulo:.2f} grados\n"
121            f"error = {self.sistema.error_minimo:.4f} m"
122        )
123        return punto, texto
124
125    animacion = FuncAnimation(
126        fig, actualizar, frames=len(t),
127        init_func=iniciar, interval=35, blit=True
128    )
129
130    ax.legend()
131    plt.show()
132    return animacion
133

```

```

134 def main():
135     velocidad_inicial = 28.0
136     gravedad = 9.81
137     objetivo_x = 5.0
138     objetivo_y = 14.0
139
140     proyectil = Proyectil(velocidad_inicial, gravedad)
141     sistema = LanzamientoParabolico(proyectil, objetivo_x, objetivo_y)
142
143     sistema.simular()
144     Analizador(sistema).mostrar()
145     Visualizador(sistema).animar()
146
147 if __name__ == "__main__":
148     main()

```

9. Resultados esperados

Al ejecutar el programa, primero se muestran resultados en consola. La salida incluye la velocidad inicial, la gravedad, la posición del objetivo, el ángulo óptimo encontrado, la desviación mínima respecto al objetivo, la altura máxima y el alcance de la trayectoria.

Con los valores usados en el programa,

$$\begin{aligned}
 v_0 &= 28,0 \text{ m/s}, \\
 g &= 9,81 \text{ m/s}^2, \\
 x_{\text{objetivo}} &= 5,0 \text{ m}, \\
 y_{\text{objetivo}} &= 14,0 \text{ m},
 \end{aligned}$$

el programa busca un ángulo que haga que la trayectoria pase cerca de ese punto. Además de la salida en consola, se abre una ventana con la gráfica de la trayectoria. En esa gráfica aparece la curva del lanzamiento, el objetivo marcado con una equis y un punto animado que se mueve sobre la trayectoria.

El objetivo de la animación no es solo decorar el trabajo. Sirve para ver de forma directa cómo el proyectil sube, alcanza una altura máxima y luego baja por efecto de la gravedad. También permite ubicar visualmente el blanco y entender si la trayectoria se aproxima a él.

10. Análisis de resultados

El resultado principal del proyecto es el ángulo óptimo. Este ángulo depende de la velocidad inicial, de la gravedad y de la posición del objetivo. Si se cambia alguno de esos datos, el programa puede encontrar un ángulo diferente.

La trayectoria obtenida tiene forma parabólica, como se espera en un movimiento ideal bajo gravedad constante. En el eje horizontal el proyectil avanza de forma uniforme, mientras que en el eje vertical sube al comienzo y después cae debido a la gravedad. Esto coincide con las ecuaciones usadas en el marco teórico.

La desviación mínima respecto al objetivo permite saber qué tan cerca pasa la trayectoria del blanco. Esta cantidad no debe confundirse con error experimental. No se está midiendo una práctica real con incertidumbre, sino usando una medida matemática para que el programa pueda decidir qué ángulo es mejor.

La altura máxima ayuda a interpretar qué tan alto llega el proyectil, y el alcance indica hasta qué distancia horizontal llega antes de volver al suelo. Estos datos complementan el resultado principal, porque no solo se obtiene el ángulo, sino también una descripción general del movimiento.

11. Cumplimiento de la rúbrica

Requisito	Cómo se cumple en el proyecto
Al menos tres clases propias	Se usan las clases <code>Proyectil</code> , <code>LanzamientoParabolico</code> , <code>Analizador</code> y <code>Visualizador</code> .
Clase base abstracta	Se define la clase <code>SistemaFisico</code> usando ABC.
Herencia	<code>LanzamientoParabolico</code> hereda de <code>SistemaFisico</code> .
Uso de NumPy	Se usa para funciones trigonométricas, conversión de ángulos, arreglos de tiempo y valores máximos.
Uso de Matplotlib	Se usa para graficar la trayectoria, el objetivo y el punto animado.
Uso de SciPy	Se usa <code>minimize_scalar</code> para buscar el ángulo óptimo.
Datos simulados	Se generan los arreglos <code>t</code> , <code>x</code> y <code>y</code> .
Visualización	Se muestra la trayectoria parabólica y el objetivo.
Animación	Se anima el proyectil usando <code>FuncAnimation</code> .

Análisis de resultados	La clase <code>Analizador</code> imprime los resultados principales y estos se interpretan en el documento.
Código que compile	El programa está organizado en un solo archivo de Python y mantiene una estructura clara.
Diseño de solución	El problema se dividió en clases con responsabilidades distintas.
Planteamiento del problema y objetivo	Se plantea la búsqueda del ángulo para acercarse a un objetivo fijo.

12. Conclusiones

Se logró construir una simulación computacional de un lanzamiento parabólico usando un modelo físico ideal. El programa calcula la trayectoria a partir de las ecuaciones de cinemática y permite encontrar un ángulo de lanzamiento adecuado para aproximarse a un objetivo definido.

El uso de `SciPy` permitió darle una parte computacional más fuerte al proyecto, porque el programa no se limita a graficar una curva, sino que busca numéricamente el ángulo que minimiza la desviación respecto al blanco.

La programación orientada a objetos ayudó a organizar el código de una forma más clara. Cada clase cumple una función concreta: una define la estructura general, otra guarda los datos del proyectil, otra calcula la trayectoria, otra analiza resultados y otra se encarga de la visualización.

La animación también fue importante porque facilita la interpretación física del movimiento. Al ver el punto moverse sobre la curva se entiende mejor cómo cambia la posición del proyectil con el tiempo y cómo la gravedad afecta su trayectoria.